SMART INDIA HACKATHON 2026 · PS 26147

Every word in our project, explained

Our tool takes an unlabelled radio recording and works out, on its own, how the data inside it was built — then rebuilds the message. This page explains every term that appears anywhere in the project: the screen, the README, the slides, the code. *No prior radio knowledge assumed.*

CONTENTS

01 The whole idea

02 Radio and signals

03 Radio as numbers

04 Riding on a wave

05 Getting bits back

06 Frames and packaging

07 Surviving noise

08 Working it out blind

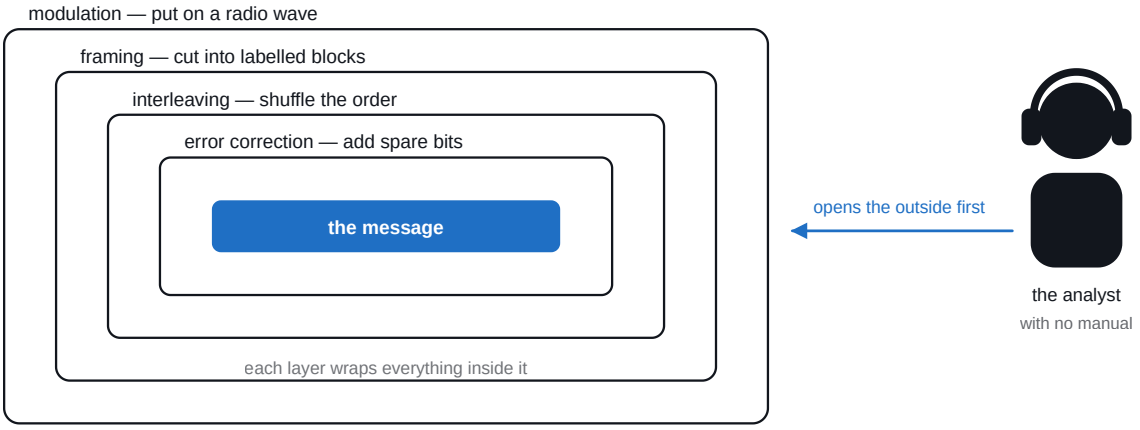
09 Software words

10 Screen decoder ring

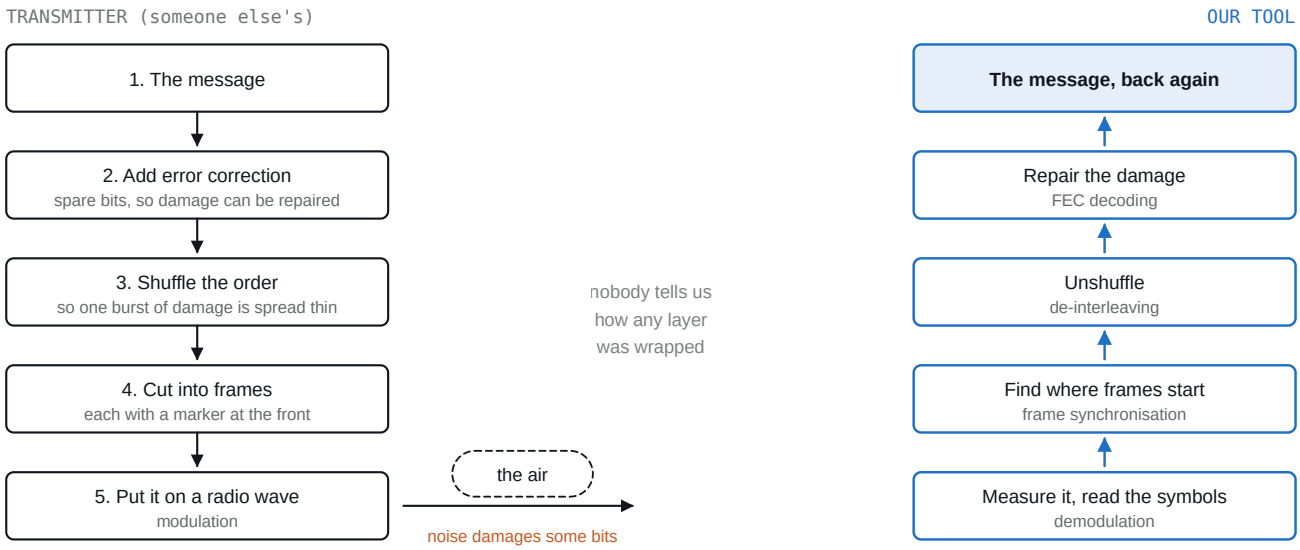
01

The whole idea, in one picture

Before any single word: here is what is going on. A transmitter takes a message and wraps it in four layers of packaging to survive the journey through the air. Our tool is the machine that unwraps all four — without being told how any of them were wrapped.



The message is a parcel inside four wrappings. Each layer was added for a reason — to survive noise, to survive bursts of noise, to be findable, to travel through the air — and each has to come off in the opposite order. The next picture shows the same thing as a sequence.



Read down the left, then up the right. The transmitter wraps the message in four layers. Our tool unwraps them in reverse. The hard part is the text in the middle: a normal receiver is told the settings in advance, and ours has to deduce them from the recording alone.

Signal intelligence SIGINT

Learning something useful from a transmission you were not meant to receive.

A monitoring station hears a transmission but has no manual for it. Signal intelligence is the work of figuring out what it is and what it carries. Our problem statement is one slice of that: given a recording, recover the settings and the data.

02

Radio and signals

The vocabulary for describing a radio transmission before anyone worries about what is inside it.

Signal

Something that changes over time and carries information by changing.

A radio signal is an electrical wave. It is useful because someone deliberately varies it — makes it stronger, or shifts it — and those variations stand for data. A wave that never changes carries nothing.

Frequency `Hz, kHz, MHz`

How many times per second the wave wiggles.

One wiggle per second is one **hertz** (Hz). A thousand is a kilohertz (kHz), a million a megahertz (MHz). When you tune a radio to 98.3 FM you are choosing 98.3 MHz — a wave wiggling 98.3 million times a second. Frequency is the dial position.

HF, VHF, UHF

Names for three neighbourhoods on the dial.

HF (high frequency) is 3–30 MHz — shortwave, bounces off the ionosphere, reaches across oceans. VHF is 30–300 MHz — FM radio, aircraft, walkie-talkies. UHF is 300 MHz–3 GHz — TV, mobile phones, Wi-Fi. Our problem statement names all three because a monitoring station listens across all of them.

Carrier

The plain wave that does the travelling, before any data is put on it.

Data cannot fly through the air by itself. A steady wave at a chosen frequency is generated as the vehicle, and the data is added by modifying it. The vehicle is the carrier. A carrier with nothing on it is just a whistle.

Bandwidth `occupied bandwidth`

How wide a slice of the dial the signal takes up.

A signal is never at exactly one frequency. Putting data on the carrier smears it across a range — and the faster the data, the wider the smear. Bandwidth measures that width in hertz. It is the single most useful clue about a signal you know nothing about, because it puts a ceiling on how fast the data can possibly be.

IN OUR TOOL The Spectrum tab shades the occupied band. We use the 99 % definition: the width that contains 99 % of the signal's power, which avoids being fooled by faint edges.

Noise and the noise floor

The permanent background hiss that every receiver hears, even with no signal present.

Noise comes from the warmth of the electronics and of the universe itself. It can never be removed, only out-shouted. The **noise floor** is its level — the flat carpet you see on a spectrum display, out of which real signals rise as bumps.

SNR `signal-to-noise ratio`

How much louder the signal is than the hiss.

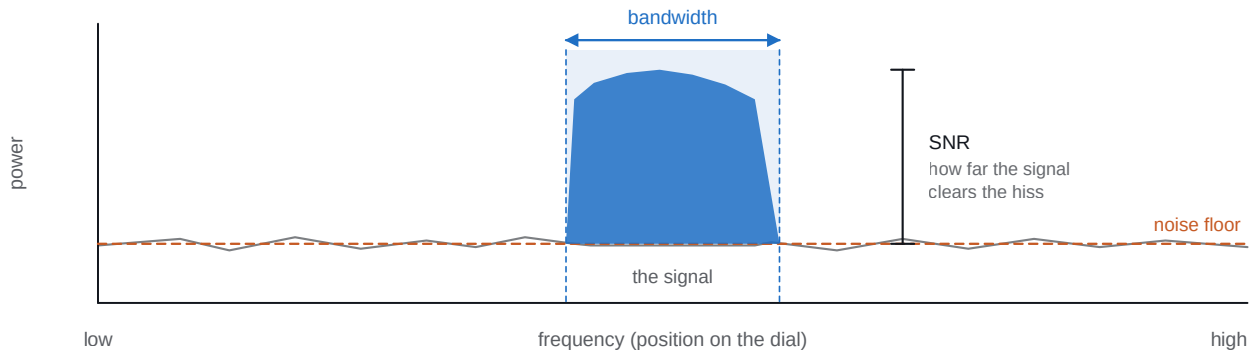
This one number decides whether anything is possible at all. High SNR and everything is easy; low SNR and even a perfect receiver fails. Every claim about a radio system is meaningless without stating the SNR it was measured at.

IN OUR TOOL Our test recordings run from 9 dB (hard) to 25 dB (comfortable). Below about 8 dB, identification stops working — which is true of any receiver, not just ours.

Decibel dB

A squashed scale for comparing two strengths, because the real range is enormous.

The faintest signal a receiver can hear and the strongest it can survive differ by a factor of billions — unusable on a normal axis. Decibels compress it: every **+3 dB is roughly double**, every **+10 dB is ten times**. So 20 dB SNR means the signal is 100× stronger than the noise, and 0 dB means they are equal.



One picture holds four of the terms above. The wobbly carpet is noise; the dashed line is the noise floor; the blue bump is the signal; its width is bandwidth and its height above the carpet is SNR. This is exactly what the Spectrum tab shows.

03

Turning radio into numbers

A computer cannot hold a wave. It holds lists of numbers. This section is about how one becomes the other — and what our input files actually contain.

Sampling and sampling rate

Measuring the wave's height many thousands of times a second and writing down each measurement.

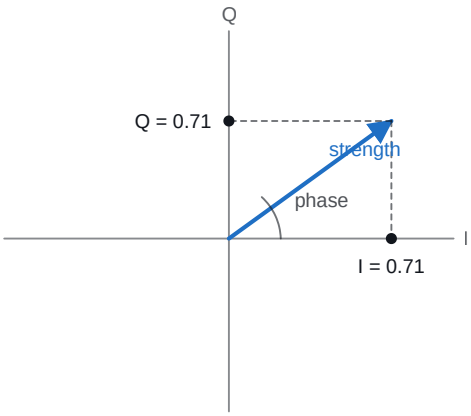
Each measurement is a **sample**. The **sampling rate** is how many are taken per second — 48,000 samples per second is written 48 kHz. Take too few and the wave cannot be reconstructed; the rule of thumb is you need at least twice the highest frequency present.

IN OUR TOOL Everything downstream is measured relative to the sampling rate, so getting it right matters more than any other single input. A `.wav` file states its own rate; a raw `.iq` file does not, which is why the app asks.

I/Q in-phase and quadrature

Storing each sample as two numbers instead of one, so that both the strength and the twist of the wave are captured.

A single number per sample records how strong the wave is, but not where it is in its cycle — and modern radio hides data in exactly that. The fix is to record two numbers: **I** and **Q**. Together they describe an arrow: its length is the strength, its direction is the phase. Every pair of numbers in an `.iq` file is one such arrow.



What the file actually holds

```
I Q I Q I Q I Q
0.71 0.71 0.99 0.10 0.02 1.00 -0.7 0.7
```

one arrow per sample, 48,000 arrows per second, nothing else

The whole job is: read the arrows, work out which of a handful of agreed directions each one meant.

I and Q are just the two shadows of one arrow. The arrow's length is how strong the signal is at that instant; its angle is the phase. Recording both is what lets a receiver see phase — and phase is where most digital radio hides its data.

Baseband and passband

Two ways of storing the same signal: centred on zero, or still sitting up at its real dial position.

A receiver normally slides the signal down so it sits around 0 Hz — this is **complex baseband**, and it is what I/Q files hold. **Passband** means the recording still sits where it was on the dial. An HF radio's audio output is a passband recording: a single stream of numbers, like any sound file.

Hilbert transform `analytic signal`

A mathematical trick that invents the missing Q from a recording that only has I.

A mono audio file has one number per sample, so the phase information seems lost. It is not: the Hilbert transform reconstructs the second half from the first. This is how our tool handles ordinary audio recorded off a shortwave receiver rather than proper I/Q.

IN OUR TOOL Sample `sig05_hf_audio.wav` is exactly this case — plain audio, converted on load, then decoded by the same pipeline as everything else.

SDR and RTL-SDR

A radio receiver that does its work in software instead of in dedicated circuitry.

A **software-defined radio** digitises a chunk of the spectrum and hands the numbers to a computer, which does the rest. The **RTL-SDR** is the famous cheap one — a USB stick built from repurposed TV-tuner chips, about ₹2,000, which made all of this a hobby rather than a laboratory activity. It writes samples as `cu8`.

Sample formats `cf32, cf64, ci16, ci8, cu8`

The different ways those I and Q numbers get written into a file.

A raw `.iq` file has no header — it is a bare wall of bytes, and you must be told how to read them. The names encode the recipe: `c` for complex (I and Q interleaved), then the number type. `cf32` is 32-bit decimals, GNU Radio's default. `ci16` is 16-bit whole numbers. `cu8` is single unsigned bytes, what an RTL-SDR produces. Read a file with the wrong recipe and you get noise.

WAV file

An ordinary sound file, which turns out to be a perfectly good container for radio.

Unlike raw `.iq`, a WAV states its own sampling rate in a header, so nothing needs to be guessed. A **stereo** WAV can carry I on the left channel and Q on the right. A **mono** WAV carries a passband recording, which needs the Hilbert transform above.

04

How data rides on a wave

Modulation is the art of changing a wave in agreed ways so the changes spell out bits. Which agreement was used is the first thing our tool has to work out.

Modulation

Changing something about the carrier wave — its strength, its timing, or its frequency — to spell out data.

There are only three properties to play with, so every scheme in existence is some combination of them. Change the strength: amplitude. Change the twist: phase. Change the pitch: frequency. Change two at once and you get more data through.

Symbol, symbol rate, baud

One "beat" of the signal. The symbol rate is how many beats per second.

The transmitter holds the wave in one agreed state for a short moment, then moves to the next. Each held moment is a **symbol**, and the count per second is the **symbol rate**, measured in **baud** (Bd). This is not the same as the bit rate: a symbol can carry more than one bit.

IN OUR TOOL `sig01` runs at 2,400 Bd — 2,400 beats per second. Since it is BPSK (one bit per beat), that is also 2,400 bits per second.

Bits per symbol

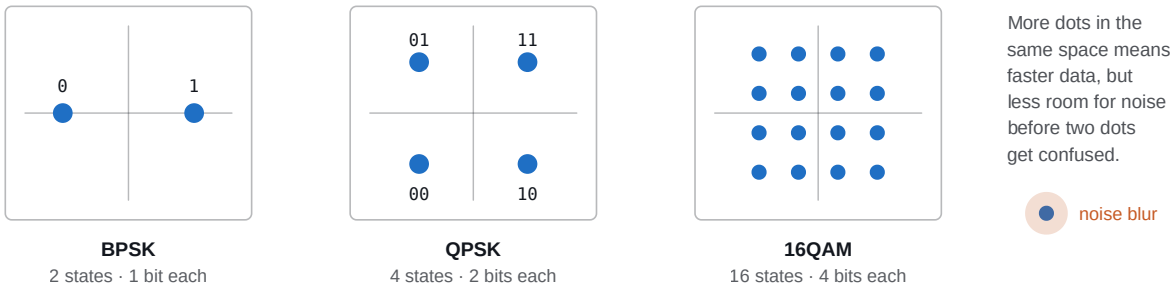
How many bits each beat carries — decided by how many different states the scheme allows.

Two possible states means each beat answers one yes/no question: 1 bit. Four states: 2 bits. Sixteen states: 4 bits. So 16QAM moves data four times faster than BPSK at the same symbol rate. The catch is that squeezing more states into the same space puts them closer together, and noise then confuses them more easily. Every modulation choice is this trade.

Constellation `constellation diagram`

A picture of all the states a scheme allows, plotted as dots.

Draw each symbol as its I/Q arrow tip and you get a scatter of dots. A clean signal lands tightly on the agreed positions; noise smears each dot into a fuzzy blob. It is the single most informative display in digital radio — you can read the modulation, the noise level and most faults straight off it.



BPSK
2 states · 1 bit each

QPSK
4 states · 2 bits each

16QAM
16 states · 4 bits each

The three linear schemes our tool handles, plus 8PSK. Each dot is one allowed state and carries a fixed pattern of bits. The receiver's job is to decide, for each received blob, which dot was meant.

PSK `BPSK, QPSK, 8PSK`

Phase shift keying: the data is in the wave's twist, not its loudness.

BPSK uses two opposite twists — 1 bit per symbol, and extremely robust. **QPSK** uses four — 2 bits. **8PSK** uses eight — 3 bits, but the twists are now only 45° apart and noise starts to bite. PSK is everywhere in satellite and military links because amplitude is left alone, which suits amplifiers.

QAM `16QAM`

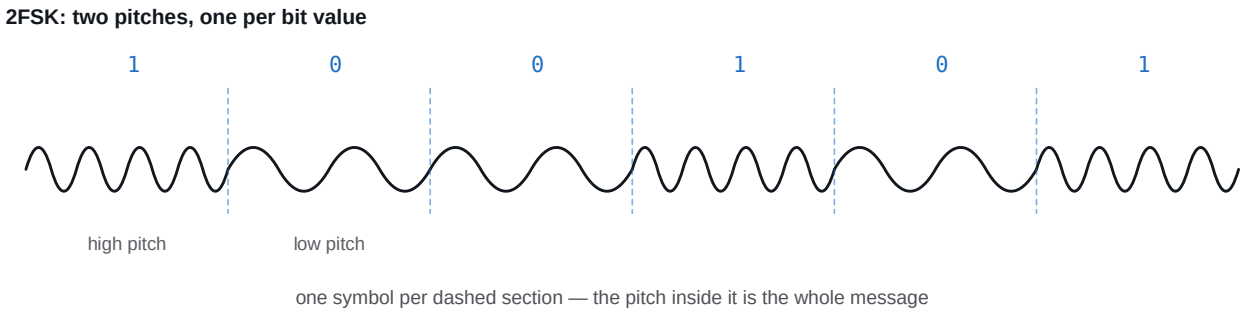
Quadrature amplitude modulation: use both the twist and the loudness at once.

PSK wastes the amplitude dimension. QAM uses it, filling the space with a grid of dots — 16 of them in 16QAM, so 4 bits per symbol. More throughput, but now the receiver must judge loudness accurately too, so it needs a better SNR.

FSK `2FSK, 4FSK, 8FSK`

Frequency shift keying: hop between two or more pitches, like a two-note whistle.

The oldest and most rugged digital scheme. 2FSK uses two tones (one for 0, one for 1), 4FSK uses four, 8FSK eight. It survives bad channels well, which is why shortwave and pager systems use it. It needs no phase tracking at all, so the receiver is simpler — our tool detects FSK by clustering the frequencies it sees into evenly spaced groups.



FSK is a whistle that changes note. Because only the pitch matters, the receiver can ignore phase entirely — which is why FSK keeps working on channels that defeat PSK.

Pulse shaping `RRC, root raised cosine, roll-off`

Rounding off the corners of each symbol so the signal does not splatter into the neighbours' channels.

Switching instantly between states would be spectrally violent — sharp edges spread energy far across the dial. So each symbol is smoothed with a shaping filter, almost always a **root raised cosine**. The **roll-off** (typically 0.35) sets how gentle the smoothing is: gentler means narrower bandwidth but fussier timing.

Matched filter

The receiver applying the same shaping in reverse, which also happens to be the best possible way to hear through noise.

There is a satisfying result in signal theory: the filter that maximises SNR is the one shaped like the pulse you are looking for. So undoing the transmitter's shaping and optimising reception are the same operation. This is why the receiver must know the roll-off.

Samples per symbol

How many measurements the recording contains for each beat of the signal.

Sampling rate divided by symbol rate. At 48 kHz sampling and 2,400 Bd you get 20 samples per symbol — twenty measurements describing each beat. More is easier to work with; fewer than about four makes life hard.

Eye diagram

All the symbol transitions drawn on top of each other, which forms an eye shape when the timing is right.

Chop the signal into symbol-length pieces and overlay them. If sampling instants are well chosen, the traces pass through a wide open gap — the "eye". A closed or blurry eye means the timing is wrong or the noise is winning. It is the classic go/no-go display, readable at a glance.

05

Getting the bits back

Between transmitter and receiver, everything drifts. These terms name the drifts and the corrections that undo them.

Demodulation

Turning the received wave back into a stream of bits.

The reverse of modulation, and the stage where all the corrections below live.

Carrier offset `frequency offset`

The transmitter's idea of the frequency and the receiver's disagree slightly.

No two crystal oscillators are identical, and movement adds Doppler shift. The result is that the whole signal sits a little off from where the receiver expects. On a constellation display the effect is unmistakable: the dots rotate steadily, smearing into a ring.

IN OUR TOOL `sig01` carries a deliberate 120 Hz offset. The "before carrier recovery" plot shows the ring; the "after" plot shows it stopped.

Clock drift `ppm`

The two ends disagree about how long a symbol lasts, so the receiver slowly slides out of step.

Measured in **parts per million**. At 20 ppm the receiver gains or loses one symbol every 50,000 — harmless for a moment, fatal over a long recording, so timing must be tracked continuously rather than set once.

Timing recovery

Working out exactly when each symbol begins, with no clock signal to help.

Nobody sends a separate tick. The receiver has to infer the beat from the data itself, and then keep following it as it drifts. Sample at the wrong instant and you read the blur between two symbols instead of the symbol.

Carrier recovery

Measuring the frequency and phase error, then rotating everything back into place.

Once the constellation stops spinning, the dots snap into tight clusters and the bits can be read. Our tool does it in two steps: a coarse estimate from the signal's own structure, then a fine tracking loop.

Phase ambiguity `rotation, spectrum flip`

The receiver can lock onto the constellation but have it turned the wrong way round, like a map with no compass.

A QPSK constellation looks identical when rotated by 90°, so recovery may settle on any of four orientations — three of which decode to nonsense. There is no way to tell from the signal alone. The fix is to try every rotation and see which one makes the known preamble appear.

IN OUR TOOL The Frame sync tab reports which rotation won — "rot 0 deg", "rot 90 deg" and so on. The preamble is what breaks the tie.

Hard and soft decisions `LLR, log-likelihood ratio`

Instead of declaring each bit a 0 or a 1, pass on how confident you are.

A **hard decision** throws away information: a symbol that landed almost exactly on a dot and one that landed halfway between two both become a flat "1". A **soft decision** keeps the confidence — "probably 1, but only just". Error-correction decoders use that confidence, and it is worth about 2 dB of SNR, which is an enormous free gain.

BER `bit error rate`

What fraction of bits arrived wrong.

A BER of 0.01 means one bit in a hundred is flipped. **Channel BER** is measured before error correction — the raw damage. After correction it should be zero. Comparing the two is how you show the error correction is earning its keep.

06

Frames: how the bits are packaged

A long stream of bits is useless if you do not know where anything starts.
Transmitters solve this by cutting the stream into labelled blocks.

Bitstream

The raw string of ones and zeros coming out of the demodulator, with no punctuation.

Like a page of text with no spaces, capitals or full stops. Everything in this section is about restoring the punctuation.

Frame

One block of the stream, with a marker at the front so the receiver can find it.

Frames repeat at a fixed length, which is itself a clue: if a receiver can find the length, it can find every boundary.

Preamble

A fixed, known pattern at the start of every frame that says "here I am".

It carries no data. Its job is to be findable — which means being unlike everything around it. Good preambles are designed so that they only match themselves and no shifted version, which makes the detection spike sharp.

IN OUR TOOL A 48-bit pattern, `C8B3A9F43893`, chosen for exactly that property.

Sync word `ASM, attached sync marker, 1ACFFC1D`

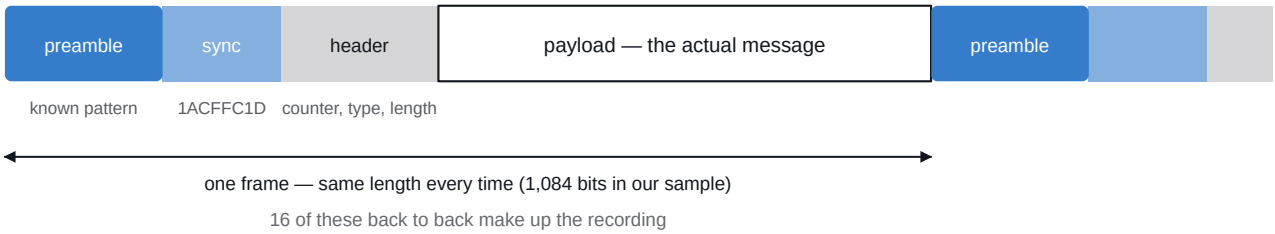
A second known pattern, this time inside the decoded data, marking the true start of the frame's contents.

The hex value `1ACFFC1D` is not arbitrary: it is the marker specified by **CCSDS**, the international body whose standards nearly every space agency uses for telemetry. Using it means our frames look like real satellite frames rather than something we invented.

Header and payload

The label on the parcel, and the contents.

The **header** holds housekeeping: a frame counter, a type code, a length. The **payload** is the actual message. Separating them is the last step before the data is readable — and a receiver that does not know the format has to deduce where the boundary falls.



The structure a receiver has to rediscover. The preamble is found first, which reveals the frame boundaries; the sync word then confirms the contents start where expected; the header is peeled off to leave the payload.

Correlation `cross-correlation`

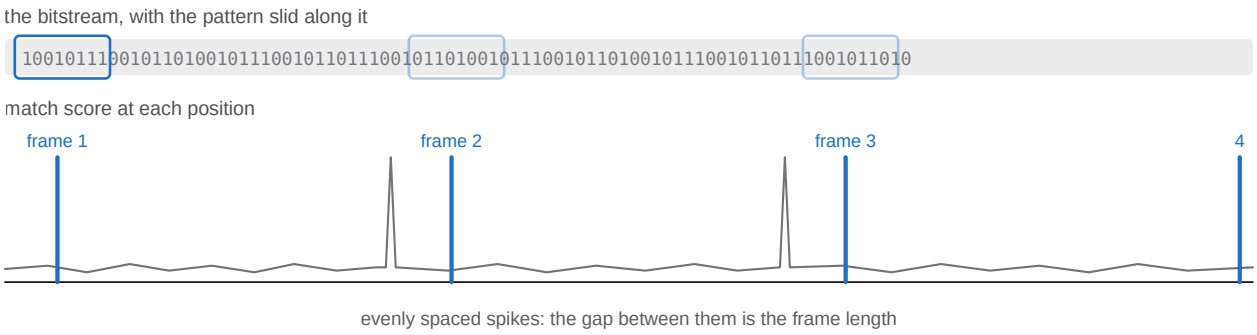
Sliding a known pattern along the data and scoring the match at every position.

Where the pattern is present the score spikes; everywhere else it stays low. This is how a preamble is found, and it works even when several bits of the preamble were received wrongly — which is exactly what you need in noise.

Autocorrelation

Sliding the data against *itself* to find how often it repeats.

If something repeats every 1,084 bits, the score spikes when the data is shifted by 1,084. This finds the frame length when the preamble is unknown — you do not need to know *what* repeats to notice *that* something does.



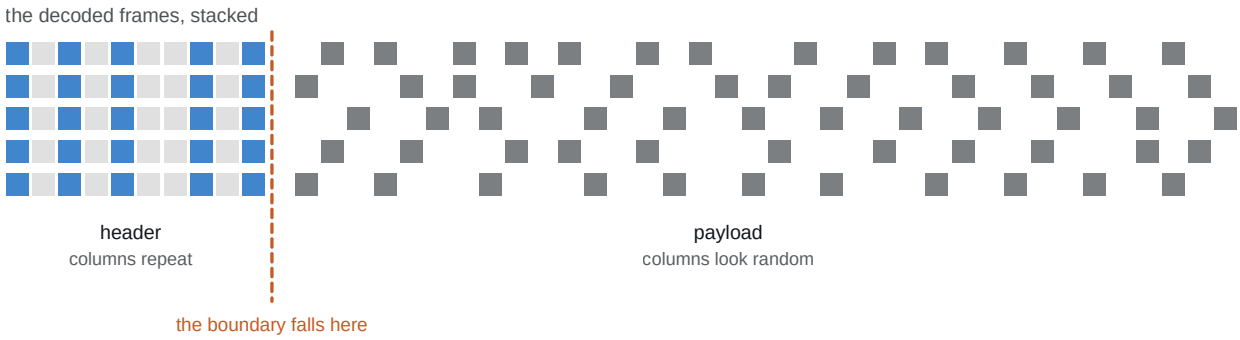
Correlation turns "find the pattern" into "find the tall spikes". The Frame sync tab in our tool shows precisely this plot; sixteen evenly spaced spikes means sixteen frames were found.

Bit constancy

Stack all the frames and ask, for each position, whether that bit is always the same.

Header positions barely change from frame to frame — the type and length are fixed, the counter changes only in its lowest bits. Payload positions look random. So the point where constancy collapses is the header/payload boundary, found without knowing anything about the format.

IN OUR TOOL This is how the Header/payload tab places the boundary at bit 70 for `sig01`, with no prior knowledge of the frame layout.



Each row is one frame; each column one bit position. On the left the columns are identical frame after frame, because that is housekeeping. On the right they are random, because that is real data. The changeover marks the boundary — no knowledge of the format needed.

Blind framing

Finding the frame boundaries when even the preamble is unknown.

Our tool has two modes. Given the preamble, it correlates for it directly. Given nothing, it finds the repeat length by autocorrelation, then finds which bits stay constant across frames — those are the preamble — and confirms the guess by checking that the sync word lands where it should. Slower, but it needs no prior knowledge at all.

07

Surviving noise: coding and interleaving

The air damages bits. Rather than ask for retransmission — often impossible — transmitters send extra information that lets the receiver repair the damage itself.

FEC `forward error correction`

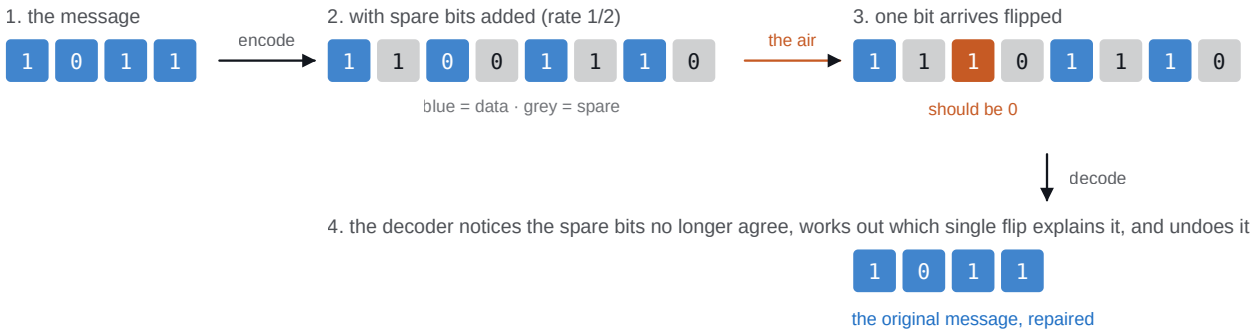
Sending carefully chosen spare bits so the receiver can fix errors without asking for anything to be resent.

"Forward" means the repair information travels forward with the data. A satellite cannot wait for a retransmission request, so this is the only option. The price is bandwidth: the spare bits take up room.

Code rate `r = 1/2`

How much of what is sent is real data.

A rate of 1/2 means half the transmission is spare bits — send two bits for every one of data. Lower rate, stronger protection, slower throughput.



Redundancy is what makes repair possible. Because the spare bits are computed from the data, a damaged bit makes the set inconsistent — and usually only one small change restores consistency. That change is the repair.

Convolutional code `K=3, K=5, K=7, (171,133)`

A code that makes every output bit depend on the last few input bits, so the data is smeared over time.

K is the **constraint length** — how many bits of memory the encoder has. K=7 means each output depends on the last seven input bits: stronger protection, but the decoder does eight times the work of K=3. The numbers in brackets, like `(171, 133)`, are the recipes for combining those seven bits, written in octal. They are published standards, not arbitrary.

Viterbi decoding

Finding the sequence of bits most likely to have produced what was received, efficiently.

A brute-force search over every possible message is impossible — the count doubles with every bit. Viterbi's insight (1967) is that you can walk through the possibilities keeping only the best path to each state, which makes the cost grow linearly instead of exponentially. It is one of the genuinely famous algorithms, and it is why convolutional codes are practical.

Reed-Solomon `RS, GF(256), t=16`

A code that works on bytes rather than bits, and is superb at repairing damage that arrives in clumps.

Reed-Solomon (1960) treats data as points on a polynomial, so extra points let you rebuild the whole curve even if some are lost. **t=16** means up to 16 corrupted bytes per block can be repaired regardless of how badly each is mangled. **GF(256)** names the arithmetic it uses, where every byte is treated as an element with its own rules for multiply and divide. The same code protects CDs, QR codes and deep-space probes.

Concatenated coding `inner and outer code`

Two codes stacked: one for scattered errors, one for clumps.

Convolutional coding handles random single-bit errors well but fails in bursts — and when it fails, it fails in bursts. So Reed-Solomon is wrapped *around* it to mop those up. The convolutional code is the **inner** code (closest to the channel), Reed-Solomon the **outer**. This exact pairing carried the Voyager images home and is a CCSDS standard.

LDPC `low-density parity check, min-sum, belief propagation`

A modern code built from thousands of simple consistency rules, solved by letting the bits gossip until they agree.

Each parity check says "these particular bits must sum to an even number". No single check is powerful, but thousands of overlapping ones are. The decoder passes confidence messages back and forth between bits and checks — **belief propagation**, usually a simplified version called **min-sum** — and after a few **iterations** the whole thing settles on an answer. LDPC gets closer to the theoretical limit than anything before it, which is why Wi-Fi, 5G and DVB all use it.

Syndrome and Berlekamp-Massey

A quick test for "is anything wrong", and the classic method for finding exactly where.

A **syndrome** is a small calculation that comes out zero when the data is consistent and non-zero when it is not. **Berlekamp-Massey** is the algorithm that turns a non-zero syndrome into the positions of the errors. We use both as *detectors*: if a block of bits produces clean syndromes, it almost certainly really is Reed-Solomon.

Burst errors

Damage that arrives in a clump — a hundred consecutive bits ruined — rather than scattered evenly.

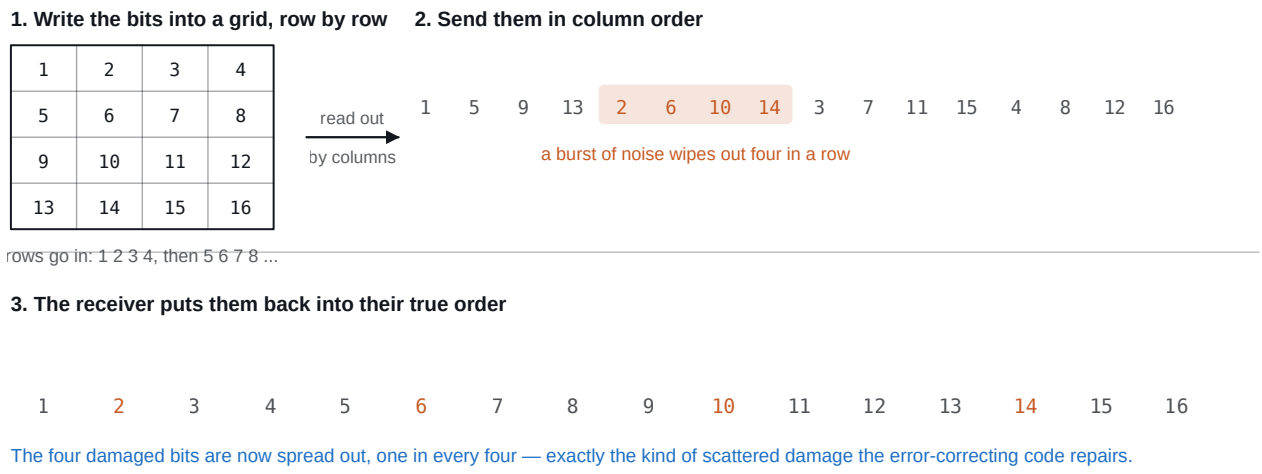
Real channels fail this way: a fade, a spark, a passing vehicle. This is a problem, because most codes can repair a few errors spread out but not many together. Which brings us to the trick in the next entry.

Interleaving `block, diagonal, convolutional, pseudo-random`

Deliberately shuffling the bit order before transmission, so a clump of damage is spread thin when unshuffled.

If 100 consecutive bits are destroyed, and the receiver unshuffles them back into their true positions, those 100 errors land far apart — one here, one there — which is precisely the kind of damage the error-correcting code can repair. The interleaver does not add any protection itself; it rearranges the damage into a shape the code can handle.

The shuffling patterns have names. **Block**: write into a grid by rows, read out by columns. **Diagonal**: read out along diagonals instead. **Convolutional (Forney)**: pass bits through a bank of delay lines of different lengths, described by two numbers, **B** (how many lines) and **J** (how much each step delays). **Pseudo-random**: a scrambled order generated from a **seed** number — unguessable unless you know the seed, which is why our tool searches through 256 of them.



Interleaving buys nothing on its own; it makes the code's job possible. Four consecutive losses would overwhelm most decoders. The same four, spread out, are routine. This is why a receiver must identify the interleaver before it can decode anything.

08

How our tool works it out blind

Everything above describes what a normal receiver is *told*. Our problem is that nobody tells us. These are the methods that deduce it from the recording alone — the part of the project that is actually novel.

Blind identification

Working out the settings from the signal itself, with no manual and no cooperation from the transmitter.

This is the whole problem statement in three words. Everything below is a tool for doing it.

Spectrum, PSD, Welch's method

A chart of how much power sits at each frequency, averaged sensibly so noise does not swamp the picture.

Power spectral density is the formal name for that chart. Computing it from one long stretch gives a jagged, unreliable result; **Welch's method** chops the recording into overlapping chunks and averages them, trading a little detail for a much steadier picture. From it we read bandwidth, carrier offset and SNR.

Waterfall

The spectrum, repeated over and over down the page, so you can see how the signal changes with time.

Frequency runs across, time runs down, and brightness is power. A steady transmission is a straight vertical stripe; a frequency-hopping one is a staircase. It is the standard display in every monitoring station.

Cyclostationarity `the symbol-rate line`

A digital signal repeats its statistical behaviour once per symbol, and that hidden rhythm can be made visible as a spike.

You cannot see the symbol rate in the spectrum directly. But apply the right non-linear operation — squaring the signal's envelope, for instance — and a sharp line appears at exactly the symbol rate. It is a genuinely clever result: a hidden periodicity forced into plain view.

M-th power test

Raise the signal to a power and see whether a tone pops out — which power works tells you the modulation.

Squaring a BPSK signal produces a clean tone; a QPSK signal needs a fourth power; 8PSK an eighth. So trying each power and seeing where the tone appears identifies the scheme. Both this and the line above also reveal the carrier offset, for free.

C42 cumulant

A statistical measurement that separates two schemes the tests above cannot tell apart.

QPSK and 16QAM both respond to the fourth-power test, so something else must distinguish them. The C42 cumulant is a number computed from the distribution of the symbols; it comes out differently for the two, because 16QAM uses several amplitude levels and QPSK only one. We correct it for the measured noise level before deciding, since noise skews it.

k-means clustering

Given a scatter of measurements, find the natural groups — here, the FSK tones.

For FSK the question is how many distinct pitches are in use. k-means is a standard algorithm that sorts values into a requested number of groups; trying 2, 4 and 8 and seeing which fits best identifies 2FSK, 4FSK or 8FSK.

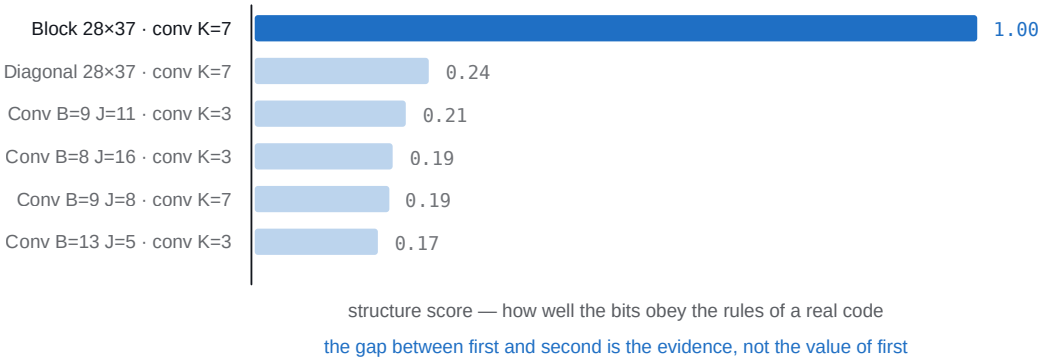
Hypothesis and structure score

Every possible "maybe it was this interleaver with that code" is a hypothesis, and each gets a score for how well the result holds together.

Here is the core trick of the project. A real error-correcting code leaves a mathematical fingerprint: with a convolutional code, a particular sum of the bits always comes out zero; with Reed-Solomon, the syndromes come out clean; with LDPC, the parity checks are satisfied. Those relations hold *only* if you guessed the interleaver and the code correctly. Guess wrong and the bits are effectively random, so the relations fail and the score collapses to near zero.

So the tool applies every plausible interleaver, tests the result against every candidate code, and scores each. One combination scores near 1.0 and everything else scores near 0.2. The answer is decided by that gap, not by a threshold anyone picked.

IN OUR TOOL For `sig01`, 1,859 hypotheses are tested in about a second. The winner scores 1.00; the runner-up 0.24.



Why the answer is trustworthy. If several hypotheses scored 0.9, 0.85 and 0.8, the result would be a guess. A single 1.00 against a field of 0.2s means the structure really is there in only one arrangement.

Ground truth

The real answer, known in advance, used to check whether the tool got it right.

Because we generate our test recordings, we know exactly what went into each one. Storing those answers alongside the files lets the tool be marked automatically rather than by eye.

Regression test

A script that re-runs every test case and reports pass or fail, so a change that breaks something is caught immediately.

Ours decodes all seven samples end to end and checks the modulation, interleaver, code and recovered message against the ground truth. Seven out of seven, every time, or something is wrong.

Synthetic transmitter

Our own software transmitter, which builds test signals with known settings and realistic damage.

It performs every transmitter stage — encode, interleave, frame, modulate — then adds noise, carrier offset and clock drift to imitate a real channel. This is what lets us test all 42 combinations of modulation, code and interleaver without owning a radio.

09

Software words

The tools the project is built from, and a few terms from the code that come up in questions.

NAME	WHAT IT IS	WHAT WE USE IT FOR
Python	A programming language	Everything. No other language in the project.
NumPy	Fast arithmetic on large lists of numbers	Every signal-processing step. A recording is a NumPy array.
SciPy	Scientific toolbox built on NumPy	Filters, the spectrum, the Hilbert transform, reading WAV files.
Streamlit	Turns a Python script into a web page	The entire interface. No HTML or JavaScript written by us.
Plotly	Interactive charts	The spectrum, waterfall, constellation, eye diagram and bar charts.
reedsolo	A Reed-Solomon library	Only the RS encode and decode. Everything else is ours.
reportlab	Builds PDF files	The one-click analyst report.

Cache

Remembering the result of slow work so it does not have to be repeated.

Analysing a recording takes a second or two. Once done, the result is kept, so clicking between tabs is instant and re-opening the same file costs nothing.

Manifest

A small file listing the true settings of every test recording.

Our `manifest.json` is the ground truth in written form — what the regression test marks against.

Pipeline

The chain of stages a recording passes through, in fixed order.

Load, measure, find the symbol rate, identify the modulation, demodulate, find frames, identify the interleaver and code, decode, split header from payload. One function runs all nine and keeps a log of what each one found.

10

Decoder ring for our screen

Every label on the interface, in plain words — useful if someone points at the screen and asks what something means.

ON SCREEN	IN PLAIN WORDS
Sampling rate	How many measurements per second the file holds.
Symbol rate	How many beats per second the signal runs at.
Samples per symbol	How many measurements describe each beat.
Occupied bandwidth	How wide a slice of the dial the signal uses.
Carrier offset	How far the signal sits from where we expected it.
Spectral SNR	How far the signal rises above the background hiss.
Decision SNR	The same idea, measured from how tightly the symbols cluster.
Classifier confidence	How sure the tool is about the modulation it named.
Frames found	How many blocks of data were located.
Frame length	How many bits are in each block.
Phase / ambiguity	Which rotation of the constellation turned out to be the right way up.
Framing mode	Whether we used the known preamble, or found the frames blind.
Hypotheses tested	How many interleaver-and-code combinations were tried.
Identification score	How perfectly the winning guess fits — 1.00 is a perfect fit.
Coded block	How many bits each frame holds once the spare bits are included.
Pad bits	Filler at the end of a frame so the length comes out even.
Channel BER before FEC	How many bits the air damaged, before repair.
RS symbols corrected	How many bytes Reed-Solomon repaired.
LDPC iterations	How many rounds of gossip the LDPC decoder needed to settle.
Bit constancy	How often each bit position stays the same across frames.

ON SCREEN	IN PLAIN WORDS
Sync match	How well the sync word matched — 1.00 is exact.

If you remember only three things

One. A transmitter wraps a message in four layers — error correction, shuffling, framing, modulation — and a normal receiver is told how all four were done. Ours is told nothing.

Two. The way we get away with that is that a real error-correcting code leaves a mathematical fingerprint that survives only if every guess is right. So we try thousands of guesses and look for the one where the fingerprint appears.

Three. The evidence is the gap, not the score. One hypothesis at 1.00 with the next at 0.24 means the structure exists in exactly one arrangement, which is a much stronger claim than passing a threshold somebody chose.